

Module 00 : Préambule & Structure du Projet

👥 Démarche et Accompagnement

Ce projet de simulation d'une dynamique gravitationnelle a été réalisé dans le cadre du TP d'algorithmique. Afin de franchir la barrière entre la physique pure et le développement logiciel, l'équipe a bénéficié de l'accompagnement d'un programmeur. Cet encadrement a servi de passerelle pédagogique : il a aidé les étudiants à appréhender l'informatique sous un angle professionnel, à acquérir la méthode de réflexion d'un ingénieur logiciel (modularité, cycle de vie de la donnée, gestion mémoire) et à structurer proprement leur code.

🤖 Usage de l'IA

L'Intelligence Artificielle a été utilisée comme un assistant technique ciblé, uniquement sur deux axes :

- **La Documentation** : Mettre en forme et condenser les choix de conception de l'équipe (format *Le Labo / Arbitrage*).
- **L'Habillage Graphique** : Configurer les détails visuels de l'interface sous Matplotlib (HUD de télémétrie, style des grilles), évitant de longs tâtonnements sur l'affichage pour se concentrer sur l'algorithmique.

📊 La Structure des Modules (Le Master Plan)

Ce tableau présente le découpage fonctionnel du logiciel et sert de table des matières pour l'ensemble de la documentation technique.

Module	Mission Critique	Défi Technique Résolu
01 : L'Entrée	Standardiser le format d'échange.	Contrat d'interface strict et générateur de configurations chaotiques.
02 : Le Parsing	Charger les données en mémoire.	Stratégie de "Double Passe" pour pré-allouer NumPy sans saturer la RAM.
03 : La Config	Interfacer l'humain et auto-piloter.	Saisies résilientes et calcul autonome du pas de temps (dt) via le temps dynamique (t_{dyn}).
04 : Le Moteur	Calculer les forces à l'instant T.	Vectorisation par <i>broadcasting</i> NumPy, neutralisation de la diagonale via <code>np.inf</code> et interception des collisions.
05 : Le Temps	Générer l'avenir du système.	Implémentation de l'intégrateur de Velocity-Verlet (symplectique) et flux d'écriture continu.
06 : Le Visuel	Restituer la cinématique.	Animation fluide à 60 FPS via la technique du <i>Blitting</i> et caméra adaptative.
07 : L'Analyse	Extraire la valeur scientifique.	Validation empirique : vitesse de libération, chaos (3 corps) et lois de Kepler.

Conclusion

Ce plan directeur cartographie la manière dont chaque brique s'assemble pour former un écosystème cohérent et robuste. Les bases architecturales étant posées, analysons le contrat qui régit notre univers virtuel.

👉 Passe au [Module 01 - Élaboration du Système d'Entrée](#).

Introduction

Pour qu'une simulation gravitationnelle puisse se faire, le programme doit avoir accès à un **système initial** sur lequel appliquer les lois de la physique. C'est le point de départ indispensable : on ne peut pas calculer un mouvement si l'on ne sait pas *qui* bouge et *d'où* il part.

[1.1] : La Carte d'Identité des Astres

Ressources >

Ref Sujet : Créer un système de départ composé d'un ensemble de corps célestes (masses, vitesses et positions initiales).

Ref Physique : Loi de la gravitation universelle de Newton :

$$F_{AB} = G \frac{m_A m_B}{r_{AB}^2}$$

Contexte : Isoler les variables physiques minimales nécessaires pour que l'univers "existe" mathématiquement dans la simulation.

Le Labo (Réflexions & Arbitrage) >

Analyse : La formule de Newton et les principes de la dynamique nous imposent des besoins précis :

1. La Masse (m) : Grandeur scalaire qui génère la force d'attraction.

2. Position (x, y) : Coordonnées nécessaires pour situer les corps dans l'espace.

3. Vitesse (vx, vy) : Sans vitesse initiale, deux astres s'effondreraient simplement l'un sur l'autre par pure attraction.

4. Le Nom : Crucial pour le post-traitement et l'identification des trajectoires (ex: distinguer la Terre de la Lune).

• Anticipation : Pourquoi pas la 3D ? Le sujet conseille de travailler en 2D car la visualisation est plus aisée. On s'arrête donc à deux coordonnées.

Analyse de la Durée : Est-ce une propriété de l'astre ?

• Réflexion : Non. Un astre existe indépendamment du temps qu'on décide de l'observer. La durée est une contrainte imposée par l'expérimentateur.

• Arbitrage : On exclut la durée de la définition des astres. Elle sera traitée comme une **Variable Globale d'Expérience** saisie lors de l'exécution.

Décision :

- Validation d'un profil d'astre composé de 6 variables : Nom , Masse , X , Y , VX , VY .
- Durée de la simulation devra être renseigné au lancement du programme

L'Émergence >

Nous connaissons maintenant la structure d'un astre isolé. Cependant, pour simuler un système complexe (N-corps), nous devons manipuler une liste arbitraire d'objets.

• Comment permettre à l'utilisateur de fournir ces données sans que cela soit une corvée ou une source d'erreurs ?

• Comment stocker cette collection de manière persistante ?

→ Voir 1.2 (Le Support de Données).

[1.2] : Le Choix du Support (Format du Fichier)

Description : Déterminer le support de transfert de données le plus efficient entre l'utilisateur et le moteur de calcul.

Le Labo (Réflexions & Arbitrage) >

Analyse des options :

• Saisie Console : Inefficace pour plusieurs corps + non persistante. Trop de risques d'erreurs de frappe.

• Format JSON / XML : Très robuste pour le code, mais complexe à rédiger pour un utilisateur non-développeur.

• Format CSV (Tableur) : Permet d'utiliser des logiciels familiers comme Excel. C'est un format "ligne par ligne" qui correspond parfaitement à notre liste d'astres.

Décision : Choix exclusif du format .csv. C'est le média qui minimise la friction utilisateur tout en restant simple à interpréter (parser).

L'Émergence >

Le contenant (CSV) est choisi. Mais l'organisation interne du fichier doit être rigoureuse : l'ordinateur doit savoir quel nombre correspond à quelle variable physique

→ Voir 1.3 (Le Contrat d'Interface).

[1.3] : Le Contrat d'Interface (Unités et Standardisation)

Ressources >

Ref Sujet : "Il est indispensable de maîtriser les unités dans ce projet".

Ref Système : Utilisation impérative du Système International (SI) : kg, m, s.

Description : Établir les règles de "langage" communes entre l'utilisateur et le programme pour garantir la cohérence des calculs physiques.

Le Labo (Réflexions & Arbitrage) >

Analyse du Header (En-tête) : Pour que le programme récupère les informations sans ambiguïté, l'ordre des colonnes doit être fixé une fois pour toutes.

Analyse des Risques :

• Unités disparates : Nous refusons les conversions complexes (UA, km, masses solaires) pour éviter les erreurs d'échelle. Le SI offre un cadre de calcul direct avec la constante G.

- **Formatage numérique** : Python utilise le point `.` comme séparateur décimal. Puisque le CSV utilise déjà la virgule `,` pour séparer les colonnes, le point est la seule alternative viable.

Décisions :

1. Fixer un en-tête obligatoire : `'Name,Mass,Position X,Position Y,Velocity X,Velocity Y'`
2. Imposer le **Système International (SI)** strict.
3. Imposer le **point** `.` pour les nombres décimaux.

[1.4] : Le Générateur de Chaos Aléatoire

Contexte : Permettre la création instantanée d'un univers à N corps totalement aléatoire afin de tester les limites de performance du moteur matriciel (NumPy) et d'observer des dynamiques chaotiques à grande échelle sans dépendre d'une saisie manuelle fastidieuse.

🔗 Ressources >

- **Modules Python** : `csv` pour l'export, `random` pour le tirage des variables, `sys` pour la gestion des arguments CLI.
- **Contrat d'interface** : Respect strict des colonnes attendues par le parseur principal (`Name` , `Mass` , `Position X` , `Position Y` , `Velocity X` , `Velocity Y`).

🧪 Le Labo (Réflexions & Arbitrage) >

Le piège de la distribution linéaire des masses :

Si nous générons des masses de manière purement linéaire (ex: `random.uniform(1e22, 1e30)`), la probabilité mathématique va saturer l'univers d'étoiles supermassives. Les petits corps (type astéroïdes ou planètes) seraient inexistants, car l'écart d'échelle est de 8 ordres de grandeur. Un tel système s'effondrerait sur lui-même de façon trop uniforme.

La solution (L'échelle logarithmique) : Nous faisons le choix d'un tirage uniforme appliqué à l'exposant de la puissance de 10 (`10**random.uniform(22, 30.5)`). De cette manière, nous assurons une distribution sémantiquement hiérarchique : le système comportera équitablement une multitude de corps légers et quelques monstres gravitationnels massifs, ce qui est idéal pour observer du chaos pur.

L'absence de contraintes orbitales :

Contrairement à un système solaire stable nous cherchons ici le déséquilibre. Les positions et les vecteurs vitesses sont injectés de manière purement cartésienne dans un carré de 40 UA de côté. Cela garantit un système hors-équilibre, propice aux collisions immédiates, aux frondes gravitationnelles sauvages et aux éjections à haute vitesse.

Décisions :

- Utilisation de la distribution logarithmique pour le calcul de la masse de chaque astre.
- Le script accepte un nombre d'astres en argument (`python generateur.py 150`)

Conclusion

Le média de transfert est prêt. L'utilisateur a ses instructions. Maintenant, le rôle du développeur commence : comment récupérer ce fichier, valider sa robustesse (gérer l'erreur humaine) et l'initialiser dans le programme ?

👉 **Direction : Module 02 - Parsing et Robustesse.**

Introduction

L'utilisateur sait désormais quoi créer et comment le faire (via le README), et son fichier CSV source est prêt sur le disque dur. L'objectif de ce module est de franchir la barrière physique : extraire les données initiales de la simulation et les transformer en variables au sein de notre programme afin de pouvoir les utiliser et appliquer la simulation.

🔗 Objectif du Module

Pour réaliser une transition parfaite entre le stockage externe (texte) et la mémoire vive (calcul), cela implique une double mission :

- 1. **L'Audit de Sécurité** : Intercepter 100% des erreurs humaines avant le calcul (typage, logique physique, doublons, colonnes manquantes) et informer l'utilisateur avec précision.
- 2. **L'Optimisation Structurelle** : Stocker ces informations dans le format le plus pertinent pour garantir l'efficacité des futurs calculs physiques.

[2.1] : Le Ciblage du Fichier (L'Aiguilleur)

Contexte : Avant d'ouvrir et de valider le contenu du fichier, le programme doit savoir où le chercher sur le disque dur et comment l'indiquer de la manière la plus fluide possible pour l'utilisateur.

📁 Ressources >

Ressources : Module sys de Python pour la gestion des arguments systèmes.

🔧 Le Labo (Réflexions & Arbitrage) >

Problématique Globale : Quelle est la méthode la plus ergonomique et robuste pour permettre à l'utilisateur de transmettre le chemin du fichier source au programme ?

Analyse des Options :

- **Option A (Chemin codé en dur dans le script)** : Oblige l'utilisateur à modifier le code Python ou à renommer continuellement ses fichiers. Inacceptable car on ne veut pas qu'il touche au code source.
- **Option B (Prompt input() pur)** : L'utilisateur tape le chemin dans la console. C'est fastidieux, propice aux fautes de frappe et surtout, on perd l'auto-complétion (la touche Tabulation du terminal). Cependant, cela reste un filet de sécurité indispensable si le programme n'est pas lancé via un terminal.
- **Option C (Arguments CLI via sys.argv)** : L'approche la plus professionnelle. L'utilisateur lance python main.py mon_systeme.csv . C'est rapide, permet l'auto-complétion et l'automatisation. S'il y a plusieurs arguments, on sécurise en ne gardant que le premier.

Arbitrage de la "Voie Hybride" : On combine le meilleur des mondes. Le programme vérifie d'abord s'il y a un argument système (Option C). Si oui, il le prend. Sinon, il déclenche le prompt input() (Option B).

Le Pare-Feu initial : Peu importe la méthode de saisie, on valide les 4 derniers caractères pour s'assurer que c'est bien un .csv avant de lancer la moindre fonction de lecture.

Décisions :

- Création de la fonction get_file_path() appliquant la méthode de sélection hybride (CLI prioritaire ou Prompt).
- Rejet immédiat avec levée d'erreur (exception) si l'extension n'est pas .csv .

⚠️ L'Émergence >

Le programme sait désormais exactement où chercher le fichier sur le disque dur, et il a la garantie que ce fichier se présente comme un CSV.

🔑 **La problématique suivante s'enclenche** : Maintenant que le fichier est localisé, comment l'ouvrir de manière sécurisée et en extraire intelligemment les données ?

→ **Passe au Maillon 2.2 (L'Extraction Automatisée - Le Lecteur)**.

[2.2] : L'Extraction Automatisée (Le Lecteur)

Contexte : Ouvrir le fichier de manière sécurisée et définir l'outil capable d'en lire les données intelligemment.

📁 Ressources >

- **Héritage** : Fichier .csv ciblé au 2.1.
- **Contrainte technique** : Gestion de la RAM et des flux d'Entrée/Sortie (I/O).

🔧 Le Labo (Réflexions & Arbitrage) >

Analyse du Problème Matériel : Si le programme plante à n'importe quel moment de la lecture, le fichier texte risque de rester "ouvert" dans la RAM, causant des problèmes de verrouillage côté système d'exploitation.

Solution : Utiliser le gestionnaire de contexte with open(...) pour déléguer la fermeture propre du fichier à Python, quoi qu'il arrive (même en cas d'erreur fatale).

L'Outil de Lecture : L'utilisation de csv.DictReader reste la meilleure option mécanique. Il lit automatiquement la première ligne comme en-tête et nous permet d'accéder aux données via des clés littérales (ex: row['mass']) plutôt que par des index (ex: row[1]). Cela rend le code infiniment plus lisible pour la maintenance.

Décisions :

- Utilisation du gestionnaire de contexte with open(...) pour l'Entrée/Sortie.
- Utilisation de csv.DictReader pour une extraction lisible.

⚠️ L'Émergence >

Le fichier est ouvert et le lecteur est prêt. Mais avant de stocker quoi que ce soit, nous devons définir les règles strictes qui sépareront les données valides des erreurs humaines (fautes de frappe, incohérences physiques).

→ **[2.3] : Gestion des Erreurs**.

[2.3] : Gestion des erreurs

Contexte : Auditer chaque donnée pour protéger l'intégrité de la simulation.

📖 Ressources >

- **Héritage :** Le contrat d'interface issu du [Module 01](#).
- **Concept Informatique :** Principe du *Fail Fast* (Échouer tôt et de manière explicite pour éviter la corruption en aval).

🔧 Le Labo (Réflexions & Arbitrage) >

Problématique Globale : Comment s'assurer que les données brutes lues dans le fichier correspondent parfaitement aux attentes mathématiques et physiques de notre programme, sans avoir à imaginer et coder chaque erreur humaine possible ?

Analyse de l'Intégrité : Si nous passons directement les données au moteur de calcul, n'importe quelle anomalie non détectée (une valeur manquante, une donnée non numérique, une aberration physique) provoquera un crash incompréhensible ou, pire, un comportement faussé à un moment indéterminé de la simulation. Essayer de lister tous les cas particuliers est une bataille perdue d'avance.

Arbitrage du Filtre Générique : Plutôt que de chercher des erreurs spécifiques, on applique un tamis strict basé sur nos contraintes :

1. **Conformité structurelle :** L'en-tête lu par le `DictReader` doit correspondre exactement à notre standard.
2. **Conformité mathématique :** Chaque donnée doit pouvoir être convertie en flottant (`float`). Le simple fait de tenter cette conversion de manière encadrée (via `try/except`) permet d'intercepter n'importe quelle donnée non numérique d'un seul coup.
3. **Conformité physique :** Une masse ne peut pas être nulle ou négative.
4. **Entité unique :** Chaque astre est identifié par un nom unique

Le Choix de l'Information Utilisateur : Au lieu de planter silencieusement, l'interception de ces anomalies nous permet de renvoyer un message d'erreur clair indiquant le numéro de la ligne fautive, rendant l'outil collaboratif et non frustrant.

Décisions :

- Création d'une constante globale définissant le standard absolu : `VALID_HEADER = ['name', 'mass', 'position x', 'position y', 'velocity x', 'velocity y']`.
- Création d'un dictionnaire centralisé `ERROR_MESSAGES`.
- Vérification immédiate de l'en-tête via `reader.fieldnames == VALID_HEADER`.
- Création d'une fonction `validate_row` avec bloc `try/except ValueError` pour le typage, et vérifications logiques (`mass > 0` , noms uniques).

⚠️ L'Émergence >

Les règles de validation sont désormais fixées et notre "pare-feu" est prêt. Cependant, avant de lancer concrètement la lecture pour filtrer et extraire ces données, nous devons préparer les réceptacles qui vont les accueillir. Sous quelle forme mathématique précise doit-on allouer la mémoire de l'ordinateur pour stocker cet univers et garantir la rapidité du futur calcul N-Corps ?

→ Voir [2.4] : L'Allocation Mémoire Anticipée. → Voir [2.4] : L'Allocation Mémoire Anticipée.

[2.4] : L'Allocation Mémoire Anticipée

Contexte : Le Module 03 (Moteur de calcul) exigera des matrices NumPy pour calculer les forces rapidement en vectoriel. Le défi de ce maillon est de trouver la méthode la plus optimisée pour convertir notre fichier texte en tableaux NumPy, tout en intégrant les règles de notre fonction qui gère les erreurs.

📖 Ressources >

- **Contrainte aval :** Le moteur physique (Module 03) calculera les forces en vectoriel.
- **Concept Informatique :**
 - Allocation mémoire contiguë (NumPy) vs allocation dynamique (Listes Python).
 - Compromis entre temps de calcul (CPU) et espace mémoire (RAM).

🔧 Le Labo (Réflexions & Arbitrage) >

Analyse de l'Approche Simple (Listes Python) : La méthode la plus intuitive et pratique serait d'ajouter chaque ligne validée dans une liste classique (`liste.append()`), puis de convertir cette liste globale en tableau NumPy à la toute fin (`np.array(liste)`). Soyons honnêtes : pour un système de 10 ou 100 astres, le temps de calcul perdu avec cette méthode est totalement insignifiant. Le véritable enjeu d'optimisation se situera dans le moteur de forces physiques.

Le Dilemme NumPy (Le Piège de la Réallocation) : Si l'on choisit tout de même d'optimiser cette étape en utilisant NumPy dès le départ pour construire un programme "scalable", on se heurte à une contrainte stricte. Les tableaux NumPy stockent les nombres dans des blocs de mémoire contigus. Si l'on tente d'agrandir ce tableau à chaque nouvelle ligne lue (ce qui équivaut à recréer un tableau plus grand et tout recopier à chaque fois), on détruit littéralement les performances de la RAM. L'optimisation devient alors pire que la méthode simple.

Arbitrage de Rigueur (La Double Passe) : Pour utiliser NumPy correctement sans tomber dans le piège de la réallocation, nous devons connaître la taille exacte des matrices *avant* de les remplir. Nous utiliserons donc une stratégie de "Double Passe".

Dilemme d'Exécution : L'Emplacement du Pare-Feu (CPU vs RAM) Puisque nous lisons le fichier deux fois, où doit-on exécuter notre validation (conversion en flottants et limites physiques) ?

- **Option A (Optimisation RAM) :** Valider dans la Passe 1. On convertit pour tester, on jette la donnée, on compte, puis en Passe 2, on reconvertit pour stocker. **Avantage :** On n'alloue la RAM que si le fichier est 100% sain. **Défaut :** On fait travailler le processeur deux fois pour rien.
- **Option B (Optimisation CPU) :** La Passe 1 ne fait que compter les lignes aveuglément. On alloue la RAM. La Passe 2 fait la conversion, le test Pare-Feu et l'injection simultanément. **Avantage :** Le processeur ne convertit le texte qu'une seule fois. **Défaut :** Si une erreur survient en Passe 2, on a alloué une matrice pour rien avant de crasher.

Arbitrage Final : Allouer temporairement quelques mégaoctets de RAM (qui seront libérés en cas de crash) n'est pas un problème. En revanche, la conversion de texte en Python est coûteuse pour le processeur. Nous choisissons donc l'**Option B (Optimisation CPU)** : Passe 1 aveugle, Passe 2 intelligente.

Décisions :

- Séparation des données finales : `names` (Liste Python), `masses` (Vecteur NumPy 1D), `positions` (Matrice NumPy 2D), `vitesse` (Matrice NumPy 2D).
- Implémentation de la "Double Passe"
- Exécution du Pare-Feu uniquement lors de la Passe 2.

Conclusion

Le passage de la barrière physique est accompli. L'univers n'est plus un texte, c'est un état mathématique chargé en RAM à $t = 0$, validé et sécurisé.

👉 La problématique suivante est le cœur du projet : comment appliquer les lois de Newton pour calculer le mouvement de ces points vers l'avenir ?

Introduction

L'univers est extrait du fichier source, converti et chargé de manière sécurisée en mémoire RAM (Module 02). Cependant, avant de lancer le moteur physique, le programme doit s'ajuster aux intentions de l'expérimentateur.

Ce module constitue la "couche d'interface" : il traduit les besoins humains (durée de l'expérience, fluidité du rendu, méthode de calcul) en paramètres mathématiques stricts exigés par l'algorithme d'intégration. L'objectif est double : offrir une interface souple et ergonomique, tout en blindant le programme avec des garde-fous algorithmiques et des fonctions d'auto-pilotage pour garantir que la simulation restera stable, même si l'utilisateur ne maîtrise pas les mathématiques sous-jacentes.

[3.1] : La Télémétrie et la Saisie (L'Interface Homme-Machine)

Contexte : Déployer un terminal interactif permettant à l'utilisateur de configurer les quatre variables globales qui guideront la boucle principale : la Durée, le Pas de temps (*dt*), l'Intervalle de sauvegarde, et le Mode de calcul.

Ressources >

- Fonctions natives** : `input()` pour la saisie, boucles `while True` pour contraindre les réponses.
- Contraintes avals** : L'Accélération (Module 04) a besoin de la variable `mode` ; L'Exportation (Module 05) a besoin de `intervalle` .

Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme 1 : L'Incompatibilité des unités (Temps Humain vs Temps Machine)

Un expérimentateur réfléchit en "Jours" ou en "Années" (ex: "Je veux simuler le système solaire sur 10 ans"). Mais la physique de Newton et notre moteur mathématique exigent strictement le Système International (les secondes).

L'Arbitrage (La Conversion Transparente) : Le terminal demande explicitement l'unité souhaitée (`J A S`). Une fois saisie, le programme convertit silencieusement la valeur humaine en seconde. L'utilisateur garde son confort de pensée, et la machine obtient sa rigueur mathématique.

Dilemme 2 : Le Tsunami de Données (Le ratio de l'Intervalle)

Si on calcule une simulation sur 10 ans avec un pas d'une seconde, on génère plus de 315 millions de positions. Tout sauvegarder saturerait le disque dur et ferait crasher le visualiseur.

L'Arbitrage (Le Rendu Standardisé) : Nous proposons un choix. Soit l'utilisateur impose son intervalle. Soit il appuie sur `Entrée` et le programme applique une règle d'ergonomie : il divise la durée totale par 1000 (`intervalle = duree / 1000`). Cela garantit un fichier CSV final de 1000 lignes.

Dilemme 3 : Le Choix du Moteur

Nous avons développé deux moteurs mathématiques (Module 04). Il faut permettre de basculer de l'un à l'autre sans toucher au code source.

L'Arbitrage : Intégration d'un sélecteur strict (1=Matrice, 2=Boucles, 3=Benchmark) qui servira d'aiguillage dans le programme principal.

Décisions Implémentées :

- Boucles de validation** : Utilisation de `try/except ValueError` couplé à des `while True` pour forcer l'utilisateur à entrer des nombres valides et empêcher le crash au menu.
- Auto-calcul de l'intervalle** : Implémentation du fallback par défaut `max(duree / 1000, 1.0)` si la saisie de l'intervalle est laissée vide.
- Sélecteur de mode** : Stockage de la décision dans la variable entière `mode` .

L'Émergence >

Les variables générales sont fixées. Le programme sait combien de temps il doit tourner et comment sauvegarder ses données.

🚩 **La problématique suivante s'enclenche** : Le terminal demande le "Pas de temps" (le *dt*, la distance temporelle entre deux calculs). Si l'utilisateur charge un univers chaotique aléatoire (Module 01), il n'a aucune idée de la vitesse des corps. S'il met un *dt* trop grand, l'algorithme va rater les courbes et les astres vont se télescoper (*Tunneling*). S'il met un *dt* trop petit, la simulation prendra des jours à calculer.

Comment le programme peut-il analyser *lui-même* la géométrie de l'univers pour calculer mathématiquement le pas de temps idéal ?

→ **Passe au Maillon 3.2 : L'Intelligence du Paramétrage**).

[3.2] : L'Intelligence du Paramétrage (La Déduction Physique du dt Universel)

Contexte : Si l'utilisateur laisse le champ du pas de temps (*dt*) vide, le système doit basculer en auto-pilote. Il doit analyser instantanément la topologie de l'univers initial (*t* = 0) pour en déduire une résolution temporelle capable de garantir la stabilité de l'intégration numérique, peu importe le scénario injecté.

Ressources >

- Analyse du Sujet (Limites)** : Le fascicule suggère un pas basé sur la vitesse maximale, une approche insuffisante si le système démarre au repos (vitesses nulles) ou subit de fortes accélérations locales.
- Concepts Astrophysiques** : Troisième loi de Kepler généralisée, temps de croisement dynamique (*t_{dyn}*), échelle de temps d'un effondrement en chute libre.
- Outils Algorithmiques** : Vectorisation par *broadcasting* NumPy pour croiser simultanément les paires sans aucune boucle `for` (Module 04).

Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme : Le Paradoxe Cinétique (Pourquoi rejeter l'analyse des vitesses initiales ?)

Pour calibrer le pas de temps *dt*, l'approche recommandée par le sujet est d'analyser la vitesse des astres.

Le problème : Cette stratégie détruit l'universalité du programme en échouant sur des cas critiques. Si un système démarre au repos (vitesses initiales nulles), l'algorithme génère instantanément une division par zéro. De plus, si un astre est proche d'un corps lourd mais possède une vitesse initiale lente, le code va déduire à tort qu'un grand pas de temps suffit, alors que l'astre va accélérer de façon fulgurante dès la première seconde, ratant sa trajectoire (effet de *tunneling*).

L'Arbitrage (L'implémentation autonome du Temps Dynamique) : Pour traiter tous les cas possibles de manière agnostique, nous choisissons d'ignorer la cinématique de départ pour nous concentrer sur le potentiel d'accélération maximal de la géométrie, en exploitant le concept astrophysique du temps dynamique d'interaction :

- L'Échelle de Temps Critique** : Pour chaque paire d'astres (*i, j*), le programme calcule son temps caractéristique d'interaction (*t_{dyn}*). Cette valeur représente le temps théorique que mettraient ces deux corps à s'effondrer l'un sur l'autre sous l'effet exclusif de leur gravité mutuelle.

$$t_{dyn} = \sqrt{\frac{r^3}{G(m_1 + m_2)}}$$

- 2. **L'Isolation du Danger** : Le script balaie la matrice des interactions et isole la valeur minimale absolue (t_{min}). C'est le point le plus instable de l'univers, le couple d'astres qui va subir la plus violente force gravitationnelle.
- 3. **La Règle des 1%** ($\eta = 0.01$) : Pour s'assurer que l'intégrateur de Verlet (Module 05) ne rate aucun virage orbital, nous convertissons ce temps en un pas de calcul verrouillé à 1% de cette crise dynamique :

$$dt = 0.01 \times t_{min}$$

Cet auto-pilotage physique décharge l'utilisateur de calculs complexes et immunise le moteur contre les variations cinétiques de départ.

Décisions Implémentées :

- **Automatisation conditionnelle** : Interception de la chaîne vide (`if not pas_input:`) pour déclencher la routine physique.
- **Calcul matriciel instantané** : Calcul de la matrice globale des temps dynamiques en réutilisant les masques de distances `r` et les sommes croisées de masses de NumPy : `np.sqrt(r3 / (G * masses_cumulees))` .
- **Neutralisation de l'auto-interaction** : Injection de `np.inf` sur la diagonale de la matrice t_{dyn} pour empêcher un corps de calculer son propre temps d'effondrement sur lui-même ($r = 0$).
- **Application du verrou de sécurité** : Extraction du minimum via `np.min()` et affectation stricte de la variable globale `pas = 0.01 * t_min` .
- **Transparence Console** : Impression immédiate de la valeur calculée dans le terminal afin que l'expérimentateur connaisse la résolution temporelle de sa simulation.

⚠ L'Émergence >

Les paramètres temporels de calcul sont désormais stabilisés et optimisés de manière autonome par rapport aux suggestions de base du sujet.

👉 **La problématique suivante s'enclenche** : Le moteur dispose de toutes ses constantes et s'apprête à générer des millions de lignes de coordonnées. Cependant, une contrainte logicielle majeure surgit : si le fichier d'export possède un nom fixe, chaque exécution va écraser les résultats précédents, anéantissant l'historique des tests et empêchant les futures analyses comparatives. Comment automatiser la gestion des fichiers sur le disque dur ?

→ **Passe au Maillon 3.3 : La Sécurisation des Données (L'Archiviste).**

[3.3] : La Sécurisation des Données (L'Archiviste)

Contexte : Gérer proprement le cycle de vie des données de sortie en concevant un mécanisme d'archivage automatisé capable de générer un fichier d'exportation unique, sans jamais exiger de saisie fastidieuse de la part de l'utilisateur ni risquer d'écraser des calculs antérieurs.

🔗 Ressources >

- **Gestion Système** : Le sous-module natif `os.path` de Python pour interagir avec le système de fichiers de manière agnostique (Windows, Mac, Linux).
- **Contrat d'Interface** : Le nom du fichier CSV source extrait au Module 02 (ex: `Inputs/chaos_3_corps.csv`).

🧩 Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme : **L'Éphémérité destructive vs Le fardeau ergonomique** Comment organiser les fichiers de sortie d'un laboratoire virtuel ?

- **Option A (Le fichier unique fixe)** : Écrire toujours dans un fichier standard `Outputs/resultats.csv` . **Défaut** : Une catastrophe pour la recherche scientifique. L'utilisateur écrase involontairement sa simulation de la veille à chaque fois qu'il lance un nouveau test.
- **Option B (La saisie manuelle systématique)** : Demander à l'utilisateur de taper au clavier un nouveau nom de fichier avant chaque calcul. **Défaut** : Ergonomie désastreuse qui brise les pipelines d'automatisation et augmente le risque d'erreur de frappe.

L'Arbitrage (Le Séquençage de Version Non-Destructif) : Nous choisissons d'automatiser entièrement la création de versions uniques par isolation sémantique. Le programme prend la racine textuelle du fichier d'entrée comme identifiant d'expérience. Il interroge ensuite le disque dur via une sonde système (`os.path.exists`) au sein d'une boucle incrémentale. Le script teste les suffixes les uns après les autres (`_S1` , `_S2` , `_S3`) jusqu'à heurter un emplacement totalement vierge. L'utilisateur ne tape rien, le programme ne détruit rien : l'historique de recherche est sanctuarisé.

Décisions Implémentées :

- **Encapsulation fonctionnelle** : Déploiement de la fonction dédiée `obtenir_nom_fichier_sortie(chemin_entree)` .
- **Isolation du radical** : Utilisation conjointe de `os.path.basename` et `os.path.splitext` pour effacer les répertoires parents et l'extension `.csv` d'origine afin de ne conserver que le nom pur de l'expérience.
- **Sonde de vérification active** : Implémentation d'une structure `while os.path.exists(...)` qui teste dynamiquement l'arborescence du dossier `Outputs/` avant de valider définitivement le chemin de sortie.

Conclusion

La phase préliminaire d'ingénierie logicielle et d'ergonomie est officiellement close.

Grâce à cette couche d'interface robuste, l'expérimentateur dispose d'un environnement de configuration hautement sécurisé : les unités de temps humaines sont converties, l'encombrement du stockage est anticipé par le découplage de l'intervalle, le pas de temps est auto-généré par l'analyse géométrique du potentiel gravitationnel (t_{dyn}), et l'archiviste garantit l'intégrité de l'historique sur le disque dur.

Toutes les variables d'état sont chargées en RAM (Module 02), les garde-fous temporels sont activés, et le protocole d'écriture est prêt. Le programme de calcul scientifique bascule maintenant dans sa phase la plus intensive : la gestion des forces.

Comment notre système va-t-il appliquer les lois de l'attraction universelle sur ces données matricielles, et comment contourner les limites de vitesse de l'interpréteur Python ?

👉 **Aller au Module 04 - Le Moteur Physique (Accélération).**

Introduction

L'univers est chargé en mémoire : nous disposons désormais des masses, des positions et des vitesses initiales de chaque corps à l'instant $t = 0$. Cependant, cet univers reste figé. L'objectif de ce module est de concevoir le "moteur" de la simulation en modélisant la loi de la Gravitation Universelle de Newton.

Au-delà de la stricte fidélité physique, ce module représente un authentique défi d'optimisation informatique : comment calculer des millions d'interactions gravitationnelles croisées à chaque pas de temps sans mettre le processeur à genoux et sans effondrer la fluidité du programme ? (temps / mémoire)

[4.1] : L'Approche Classique par Boucles (La Projection et l'Action-Réaction)

Contexte : Traduire la loi physique scalaire de Newton en coordonnées cartésiennes bidimensionnelles et implémenter une première version itérative du calcul des forces en optimisant le nombre d'opérations grâce aux symétries de la nature.

Ressources >

Lois Physiques

: Loi de la gravitation universelle ($F = G\frac{m_1m_2}{r^2}$) et Principe Fondamental de la Dynamique ($\vec{a} = \vec{F}/m$).

Outils Mathématiques

: Théorème de Pythagore pour les distances, trigonométrie des triangles semblables pour la projection linéaire.

Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme 1 : Le Découpage Géométrique (La Projection Cartésienne)

L'ordinateur travaille uniquement sur des axes orthogonaux indépendants (X et Y), tandis que la force de gravité agit selon une diagonale directe reliant deux astres. Comment répartir cette intensité ?

L'Arbitrage : Pour chaque paire d'astres, le code calcule l'écart sur chaque axe ($dx = x_j - x_i$ et $dy = y_j - y_i$), puis l'hypoténuse $r = \sqrt{dx^2 + dy^2}$. Plutôt que de calculer des angles (sinus/cosinus) avec des fonctions trigonométriques extrêmement lourdes pour le processeur, nous utilisons le principe géométrique des triangles semblables. On projette la force en la multipliant simplement par le ratio de distance de l'axe : $F_x = F \times \frac{dx}{r}$ et $F_y = F \times \frac{dy}{r}$.

Dilemme 2 : L'Explosion Algorithmique (La redondance en $\mathcal{O}(N^2)$)

La méthode la plus naïve consiste à utiliser deux boucles imbriquées complètes pour que chaque astre parcoure tous les autres. Mais si le code calcule l'influence de l'astre i sur l'astre j , il recroisera plus tard l'influence de j sur i . C'est un gâchis colossal de calculs.

L'Arbitrage (L'Action-Réaction) : Nous invoquons la 3ème loi de Newton ($\vec{F}_{ij} = -\vec{F}_{ji}$). Nous implémentons une boucle à indexation triangulaire partielle : `for j in range(i + 1, N)`. Cela résout deux problèmes d'un coup : un astre ne se compare jamais avec lui-même (évitant la division par zéro), et la force n'est calculée qu'une seule fois par paire. Elle est directement ajoutée à l'accélération de l'astre i et soustraite à celle de l'astre j , divisant le nombre d'itérations par deux : $\frac{N(N-1)}{2}$.

Décisions Implémentées :

- Architecture de calcul** : Création de la fonction de référence `calculer_accelerations_boucles(mass, pos, rayons, time)`.
- Allocation locale** : Initialisation d'une matrice d'accélération vierge via `np.zeros((N, 2))` à chaque pas de temps.
- Symétrie physique** : Application stricte du principe d'action-réaction via les opérateurs cumulatifs `+=` sur l'index i et `-=` sur l'index j .

L'Émergence >

L'astuce de l'Action-Réaction soulage l'algorithme, mais l'exécution de boucles `for` imbriquées en Python pur reste dramatiquement lente dès que le nombre de corps N augmente (langage interprété).

🚧 **La problématique suivante s'enclenche** : Comment court-circuiter totalement la lenteur de l'interpréteur Python pour déléguer le calcul de ces millions de paires directement au processeur en langage machine ?

→ **Passe au Maillon 4.2 : L'Approche Matricielle et le Broadcasting (NumPy)**.

[4.2] : L'Approche Matricielle et le Broadcasting (NumPy)

Contexte : Migrer le calcul des forces vers un paradigme vectoriel global pour exploiter la vitesse brute du langage C sous-jacent à la bibliothèque NumPy, éliminant ainsi toute boucle itérative.

Ressources >

Héritage

: Les matrices de positions 2D et de masses 1D allouées dynamiquement lors du parsing (Module 02).

Concept Informatique

: Le *Broadcasting* (extension virtuelle des dimensions matricielles sans duplication en mémoire).

Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme 1 : Le Calcul Vectoriel Simultané

Comment forcer le processeur à calculer toutes les distances inter-planétaires d'un seul coup sans utiliser de boucle ?

L'Arbitrage : Nous utilisons l'extension de dimension `np.newaxis`. En soustrayant la matrice des positions étendue en ligne à elle-même étendue en colonne (`pos[np.newaxis, :, :] - pos[:, np.newaxis, :]`), NumPy génère instantanément une matrice 3D contenant les écarts (dx, dy) de toutes les paires possibles de l'univers en une seule opération exécutée en C backend.

Dilemme 2 : Le Piège du Branching (L'Auto-interaction matricielle)

Le broadcasting croise inévitablement chaque astre avec lui-même. Sur la diagonale de notre matrice de distances, on se retrouve avec des zéros ($r = 0$), provoquant une division par zéro fatale lors du calcul de la force. Ajouter une condition classique (`if i != j`) pour sauter la diagonale forcerait le processeur à s'interrompre à chaque case pour "réfléchir" (rupture de pipeline ou *branching*). Cela briserait la vitesse du calcul vectoriel.

L'Arbitrage (La Neutralisation Algébrique) : Nous interdisons les conditions algorithmiques. Nous appliquons une ruse mathématique : nous écrasons la diagonale des zéros en y injectant la valeur de l'Infini via `np.fill_diagonal(r2, np.inf)`. On laisse ensuite NumPy appliquer la formule de Newton de manière aveugle sur tout le bloc mémoire. Comme $G\frac{m}{\infty^2} = 0$, l'auto-attraction s'annule mathématiquement d'elle-même à la vitesse maximale du processeur, sans aucune interruption de flux.

Décisions Implémentées :

- Vectorisation totale** : Implémentation de la fonction ultra-rapide `calculer_accelerations_matrice(mass, pos, rayons, time)`.
- Algèbre matricielle** : Réduction de la matrice 3D des deltas par somme quadratique sur le dernier axe (`np.sum(..., axis=-1)`) pour isoler le carré des distances `r2`.
- Verrouillage de la diagonale** : Utilisation exclusive de `np.fill_diagonal(r2, np.inf)` pour immuniser le code contre l'auto-interaction.

Le calcul vectoriel est un succès : notre moteur tourne désormais à pleine puissance, libéré des lenteurs de Python.

👉 **La problématique suivante s'enclenche** : Cette exécution purement mathématique est aveugle. Elle met en lumière un cas limite catastrophique de la formule de Newton : que se passe-t-il si deux astres finissent par se percuter ? La distance qui les sépare devient nulle. La force est divisée par zéro, explose vers l'infini, et corrompt immédiatement toute la simulation. Comment doter notre moteur d'un véritable "sens du toucher" pour détecter un contact physique avant que l'univers mathématique ne s'effondre ?

→ **Passe au Maillon 4.3 : La Singularité Gravitationnelle (Gestion et Interception des Collisions).**

[4.3] : La Singularité Gravitationnelle (Gestion et Interception des Collisions)

Contexte : Doter le moteur de calcul d'une réalité matérielle. Il s'agit d'empêcher les astres de se traverser comme des fantômes (effet de *Tunneling*) ou de générer des forces infinies, en concevant un système d'arrêt d'urgence capable de s'adapter automatiquement à l'échelle de chaque objet (de la simple sonde spatiale à l'étoile supermassive).

📖 Ressources >

- **Formules Géométriques** : Volume de la sphère ($V = \frac{4}{3} \pi R^3$) et équation de masse par la densité ($M = V \times \rho$).
- **Constante Physique** : Densité moyenne type planète rocheuse (`DENSITY = 5500 kg/m³`).

🔧 Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme 1 : L'Événement Post-Impact (Fusion, Rebond ou Arrêt ?)

Si deux astres entrent en contact physique, comment le moteur doit-il réagir ? La physique offre deux voies : la collision élastique (les astres rebondissent) ou inélastique (fusion).

L'Arbitrage (Le principe de rigueur) : Gérer une fusion exigerait de détruire et recréer dynamiquement les matrices NumPy en plein milieu de la boucle de calcul, ce qui anéantirait nos efforts d'optimisation (Module 4.2). Gérer un rebond imposerait une modélisation vectorielle complexe. Plutôt que d'implémenter une mécanique approximative, nous optons pour la sécurité absolue : le **Hard Stop**. Dès qu'un impact est avéré, le programme se gèle et lève une erreur explicite, préservant ainsi la validité scientifique de la simulation au lieu de générer des trajectoires faussées par une force infinie.

Dilemme 2 : Le Seuil de Détection (Constante Globale vs Échelle Dynamique)

Maintenant que la règle d'arrêt est fixée, il faut déterminer *quand* elle se déclenche. N'ayant pas le rayon des astres dans le fichier CSV, deux options s'offrent à nous :

- *Option A (La Constante Arbitraire)* : Fixer une limite absolue en dur dans le code pour tout l'univers (ex: si l'écart est inférieur à 5000 km, c'est un crash). **Le problème** : C'est inadapté à un système multi-échelles. Une sonde de 10 mètres s'écraserait virtuellement à des milliers de kilomètres de la Terre, et deux étoiles géantes se traverseraient presque avant que le code ne réagisse.
- *Option B (Le Rayon Numérique Déduit)* : Estimer un rayon théorique dynamique basé sur la masse de chaque corps. **L'Arbitrage** : Nous rejetons la constante globale et optons pour l'Option B. En imposant une densité moyenne universelle (`DENSITY = 5500 kg/m³`), le programme isole mathématiquement le rayon à l'initialisation : $R_i = \left(\frac{3M_i}{4\pi\rho} \right)^{1/3}$. Même si cela lisse les spécificités des géantes gazeuses, la collision devient une frontière géométrique propre à chaque paire : l'impact se déclenche si `r < R[i] + R[j]`.

Décisions Implémentées :

- **Déduction Mathématique** : Création de la sous-fonction `calculer_rayons(mass)` exécutée uniquement à l'initialisation pour générer le tableau des seuils.
- **Pare-feu d'impact vectorisé** : Génération de la matrice de collision par sommation croisée (`rayons[:, np.newaxis] + rayons[np.newaxis, :]`) permettant une vérification matricielle instantanée via `np.any(r < rayons_collision)`.
- **Interception (Hard Stop)** : Levée d'une exception explicite (`ValueError`) figeant la boucle temporelle avant l'effondrement gravitationnel.

⚠ L'Émergence >

Notre moteur physique est désormais mathématiquement complet, performant et protégé contre les collisions accidentelles. Nous disposons de deux moteurs fonctionnels dans le code (le mode Boucles et le mode Matrices).

👉 **La problématique suivante s'enclenche** : L'ingénierie logicielle exige de prouver scientifiquement l'efficacité d'une optimisation par rapport à une autre. Comment mesurer empiriquement le gain de vitesse réel apporté par le calcul matriciel ?

→ **Passe au Maillon 4.4 : Le Mode Benchmark (La Validation Empirique).**

[4.4] : Le Mode "Benchmark" (La Validation Empirique)

Contexte : Déployer un environnement de test scientifique au sein de l'application pour quantifier et comparer objectivement le temps CPU consommé par les deux méthodes de résolution (Boucles vs Matrices).

📖 Ressources >

- **Bibliothèque Système** : Le module natif `time` de Python.
- **Outil de mesure** : L'horloge haute précision `time.perf_counter()` pour capturer les microsecondes d'exécution du processeur.

🔧 Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme : L'isolation des mesures de performance Comment s'assurer que notre chronométrage mesure la pure efficacité mathématique des algorithmes sans être faussé par les lenteurs d'écriture sur le disque dur (I/O) ou par les tâches de fond du système d'exploitation ?

L'Arbitrage : Nous créons une routine d'exécution isolée (`mode == 3`). Le programme prend le même système d'entrée et lance la simulation complète deux fois consécutives. Les compteurs encadrent de manière chirurgicale les appels à la fonction de Verlet, et le script calcule lui-même le ratio mathématique final (`temps_boucles / temps_matrice`) pour l'afficher à l'utilisateur, apportant une preuve irréfutable de la supériorité vectorielle.

Décisions Implémentées :

- **Architecture de test** : Centralisation de la procédure dans la fonction dédiée `test_performance(...)`.
- **Chronométrage chirurgical** : Encapsulation des fonctions de simulation entre deux variables de capture temporelle (`debut = time.perf_counter()` ... `fin = time.perf_counter()`).
- **Restitution des métriques** : Affichage direct des résultats dans la console avec calcul automatique du vainqueur de performance pour guider les choix futurs de l'expérimentateur.

Conclusion

Notre moteur physique a maintenant deux vitesses (classique et matricielle). Nous savons maintenant calculer "l'effort" (l'accélération) que subit chaque astre à un instant T . Mais une accélération ne crée pas de mouvement toute seule : elle modifie la vitesse, qui elle-même modifie la position au cours du temps.

Comment transformer cette accélération instantanée en un déplacement réel sur une période donnée (ex: 1 jour, 1 an) ?

👉 Direction : Module 05 - L'Intégration Temporelle (Algorithme de Verlet).

Introduction

Le décor est planté, la configuration est validée (Module 03) et les lois de la physique sont modélisées (Module 04). Il est temps de lancer le temps lui-même.

Ce module constitue le cœur battant de notre simulateur. C'est ici que la physique théorique s'incarne dans une boucle temporelle itérative pour donner vie à l'Univers.

L'objectif n'est pas seulement de calculer l'avenir du système N-Corps, mais de sérialiser et d'archiver son histoire de manière fluide, performante et infaillible.

[5.1] : La Préparation du Fichier (La Stratégie d'I/O et les Métadonnées)

Contexte : Avant de calculer le premier mouvement, le programme doit ouvrir le flux de données vers le disque dur, choisir sa stratégie d'écriture, et construire une structure d'accueil capable d'enregistrer un nombre N d'astres de manière standardisée et réutilisable.

📖 Ressources >

- Héritage** : Les constantes définies au Module 03 (`duree` , `pas` , `intervalle`), le nom du fichier de sortie sécurisé, et la liste dynamique des noms des astres (`names`).
- Modules Python** : `csv` pour le formatage, `os` pour la gestion des dossiers, la fonction native `open()` pour la gestion du flux mémoire.

🔧 Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme 1 : La Stratégie d'Enregistrement (Saturation RAM vs Goulet d'étranglement I/O)

Comment sauvegarder des millions de calculs ?

- Option A (Tout à la fin)** : Stocker toutes les positions dans des tableaux NumPy géants en RAM, et créer le CSV tout à la fin. *Risque* : Saturation de la mémoire pour les longues simulations, et perte totale des données si le programme crashe à 99%.
- Option B (Au fil de l'eau)** : Ouvrir le fichier et écrire sur le disque dur à chaque étape. *Risque* : L'accès matériel au disque (I/O) est des milliers de fois plus lent que la RAM, cela briderait le processeur. *L'Arbitrage (Le Streaming Bufferisé)* : Nous choisissons l'écriture progressive (Fail-Safe). Pour éviter le ralentissement matériel, nous faisons confiance au buffer natif de la fonction `open()` de Python : les données sont poussées en continu vers le module `csv`, mais le système d'exploitation les groupe silencieusement en RAM avant de les écrire par "blocs" sur le disque. C'est le compromis parfait entre sécurité et performance.

Dilemme 2 : La traçabilité scientifique (Le fichier autoporteur)

Dans un an, si un expérimentateur ouvre notre fichier `resultats_51.csv` contenant 200 000 lignes de nombres, il n'aura aucun moyen de savoir avec quelle précision temporelle (dt) cette physique a été générée.

L'Arbitrage (Les Métadonnées masquées) : Le fichier de sortie doit contenir son propre ADN. Nous choisissons d'écrire une toute première ligne de configuration contenant explicitement les variables clés (Durée, Pas, Intervalle). Pour que cette ligne ne fasse pas crasher l'analyse de données (Pandas) par la suite, nous la préfixons d'un `#`. Le fichier devient un objet scientifique traçable et sécurisé.

Dilemme 3 : La dimensionnalité de l'en-tête (Aplatir l'Univers)

Le moteur physique gère des matrices (tableaux à plusieurs dimensions), mais un fichier CSV est une structure strictement bidimensionnelle (Lignes = Temps, Colonnes = Valeurs). De plus, l'univers peut contenir 3 astres comme 150. On ne peut pas coder les colonnes en dur.

L'Arbitrage (La génération dynamique) : Nous construisons l'en-tête du tableau de manière procédurale. Le script boucle sur la liste d'astres identifiée au parsing, et génère automatiquement pour chaque corps quatre colonnes distinctes (`Nom_X` , `Nom_Y` , `Nom_VX` , `Nom_VY`). Le tableau s'élargit mathématiquement en fonction du système initial injecté.

Décisions Implémentées :

- Flux d'écriture (Fail-safe)** : Utilisation de l'instruction `with open(..., 'w')` couplée à `csv.writer` pour initialiser un flux continu et profiter du buffer mémoire de Python au lieu de tout stocker en RAM.
- Vérification d'arborescence** : Création automatique du dossier de sortie via `os.makedirs(OUTPUT_FOLDER, exist_ok=True)` pour prévenir les crashes d'I/O.
- Injection des Métadonnées** : Écriture de la ligne `# DUREE_SEC: {duree} | PAS_SEC: {pas} | INTERVALLE_SAUVEGARDE_SEC: {intervalle}` dès l'ouverture du fichier.
- En-tête horizontal** : Utilisation d'une boucle génératrice pour concaténer dynamiquement un vecteur textuel de colonnes proportionnel au nombre d'astres (`len(names) * 4` colonnes).

⚠️ L'Émergence >

Le flux d'enregistrement est ouvert, protégé contre les crashes, et son en-tête est en place, prêt à recevoir les coordonnées de l'univers à chaque instant clé.

👉 **La problématique suivante s'enclenche** : L'univers est à $t = 0$ et le fichier attend. Il faut maintenant lancer le temps. Comment calculer la position future de nos astres avec suffisamment de précision pour que le système ne s'effondre pas sur lui-même au bout de quelques orbites ? Pourquoi l'approche classique (Euler) n'est-elle pas suffisante pour simuler la gravité sur le long terme ?

→ **Passe au Maillon 5.2 : La Précision Temporelle (L'Algorithme de Velocity-Verlet).**

[5.2] : Le Moteur d'Intégration (L'Algorithme de Velocity-Verlet)

Contexte : Transformer l'accélération instantanée subie par chaque astre en un déplacement spatial fluide et continu. Il s'agit de calculer itérativement la trajectoire du système sur des millions de pas de temps (dt) sans que la simulation ne se disloque au fil des années virtuelles.

📖 Ressources >

- Héritage** : Le calculateur d'accélération (matriciel ou par boucle) développé au Module 04.
- Concepts Physiques** : Cinématique (Position, Vitesse, Accélération), Intégrateurs Symplectiques (conservation de l'énergie géométrique).

🔧 Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme 1 : Le Drift de l'Énergie (Euler vs Verlet)

L'approche la plus intuitive pour animer un astre est la méthode d'Euler : à chaque pas, on calcule la nouvelle position avec la vitesse actuelle, et la nouvelle vitesse avec l'accélération actuelle.

Le problème : Euler fait une approximation linéaire sur une courbe elliptique. À chaque itération, l'algorithme "dépasse" légèrement la trajectoire réelle et ajoute artificiellement de l'énergie au système. Sur une simulation de 10 ans, l'orbite terrestre finirait par former une spirale et la Terre s'échapperait du système solaire.

L'Arbitrage (Le choix Symplectique) : Nous implémentons la méthode de **Velocity-Verlet**. C'est un algorithme symplectique : l'erreur numérique oscille mais ne s'accumule jamais. L'énergie globale du système est conservée, ce qui nous permet de simuler des trajectoires chaotiques ou des figures parfaites (comme l'orbite en "Huit" du Module 07) sur des décennies sans la moindre dérive.

Dilemme 2 : La séquence d'exécution (Le paradoxe temporel)

L'algorithme de Verlet est ultra-précis car il calcule la nouvelle vitesse en utilisant la moyenne entre l'accélération présente et l'accélération future. Mais pour connaître l'accélération future, il faut d'abord connaître la position future.

L'Arbitrage (L'architecture en 3 temps) : Nous imposons une séquence stricte et immuable dans la boucle de calcul (`while t <= duree`) :

1. *Le Saut spatial* : On avance toutes les planètes d'un coup grâce à leur accélération et vitesse actuelles.
2. *La Mise à jour des Forces* : On appelle notre moteur (Module 04) pour calculer les nouvelles accélération (`acc_t_next`) sur ces nouvelles positions.
3. *La Correction cinématique* : On met à jour les vitesses en utilisant simultanément l'ancienne et la nouvelle accélération.

Décisions Implémentées :

- **Rejet d'Euler** : Utilisation exclusive de l'intégration symplectique de Verlet.
- **Étape 1 (Positions)** : Application de l'équation cinématique `pos = pos + vel * pas + 0.5 * acc_t * (pas ** 2)`.
- **Étape 2 (Accélérations)** : Appel au moteur gravitationnel pour générer `acc_t_next` en fonction du mode choisi (matrice ou boucle).
- **Étape 3 (Vitesses)** : Application de l'équation de correction `vel = vel + 0.5 * (acc_t + acc_t_next) * pas`.
- **Recyclage Mémoire** : Écrasement de l'ancienne accélération par la nouvelle (`acc_t = acc_t_next`) à la fin de la boucle pour éviter de saturer la RAM avec des variables inutiles.

⚠ L'Émergence >

Les mathématiques tournent. La Terre avance autour du Soleil avec une précision chirurgicale à chaque pas de calcul (*dt*).

👉 **La problématique suivante s'enclenche** : Si l'utilisateur a choisi un pas de 1 seconde pour une simulation d'une année (31 millions de secondes), le programme va exécuter 31 millions de boucles mathématiques parfaites. Écrire chaque ligne dans notre fichier CSV générerait un fichier de plusieurs gigaoctets, inexploitable par notre visualiseur.

Comment dé-corréler le temps mathématique (le *dt*) du temps d'enregistrement (l'intervalle) sans utiliser la fonction modulo `%` qui provoque des erreurs catastrophiques sur les nombres flottants en informatique ?

→ **Passer au Maillon 5.3 : La Boucle Temporelle (Le Piège du Modulo et la Sauvegarde).**

[5.3] : La Boucle Principale et l'Extraction d'État (Sérialisation)

Contexte : Le moteur tourne et calcule les nouvelles positions à chaque pas de temps (*dt*). Il faut maintenant extraire ces données à des intervalles précis pour les écrire dans le fichier CSV, c'est-à-dire transformer un état matriciel 2D en une simple ligne de texte 1D, tout en gardant une synchronisation temporelle parfaite.

📖 Ressources >

- **Héritage** : Les variables `t`, `pas` et `intervalle`.
- **Modules Mathématiques** : Manipulation de matrices avec NumPy (`np.hstack`, `flatten()`).
- **Concept Informatique** : La représentation des nombres à virgule flottante (Standard IEEE 754).

🔧 Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme 1 : La synchronisation temporelle (Le Piège du Modulo sur les flottants)

Pour savoir quand sauvegarder une image, l'approche naïve est d'utiliser le modulo : si le reste de la division du temps par l'intervalle est nul (`t % intervalle == 0`), on sauvegarde.

Le problème : En informatique, les nombres décimaux (flottants) sont imparfaits (ex: $0.1 + 0.2 = 0.30000000000000004$). Sur des millions d'itérations, le temps *t* ne tombera presque jamais *exactement* sur un multiple parfait de l'intervalle. Le modulo ratera la condition, et le fichier sera vide.

L'Arbitrage (Le Seuil Cumulatif) : Nous abandonnons le modulo. Nous créons une variable "cible" (`prochaine_sauvegarde`). À chaque boucle, on vérifie simplement si le temps a dépassé cette cible (`if t >= prochaine_sauvegarde`). Si oui, on écrit la ligne, puis on repousse la cible (`prochaine_sauvegarde += intervalle`). C'est mathématiquement infaillible, peu importe l'imprécision des décimales.

Dilemme 2 : L'Aplatissage de l'Univers (La Sérialisation 2D vers 1D)

Notre moteur physique stocke les positions et les vitesses sous forme de deux matrices 2D distinctes (Lignes = Astres, Colonnes = Axes X/Y). Mais la fonction `csv.writer` n'accepte qu'une liste simple (un vecteur 1D) pour écrire une ligne dans le fichier.

L'Arbitrage : Plutôt que de faire des boucles `for` imbriquées très lentes pour extraire chaque coordonnée une par une, nous utilisons la puissance vectorielle de NumPy. Nous collons d'abord horizontalement la matrice des positions et des vitesses (`np.hstack((pos, vel))`), puis nous écrasons le tout en une seule dimension avec `.flatten()`. La conversion est instantanée.

Décisions Implémentées :

- **Découplage temporel strict** : Le rythme du moteur physique (calcul) est totalement indépendant de la fréquence d'enregistrement (sauvegarde) grâce au système de seuil cumulatif.
- **Sérialisation optimisée** : L'instruction `np.hstack((pos, vel)).flatten()` est utilisée pour fusionner et aplatir les matrices d'état avant l'écriture.
- **Avancement du temps** : Le temps est incrémenté strictement de `t += pas` à la toute fin de la boucle d'intégration.

Conclusion

Le temps s'est écoulé. Les planètes ont suivi leurs trajectoires selon les lois immuables de Newton et la précision chirurgicale de Verlet. L'univers entier est désormais archivé de manière sécurisée et continue dans un fichier CSV.

La simulation physique est une réussite technique : le processeur a été optimisé grâce au calcul matriciel, la mémoire a été préservée par l'écriture en flux continu (Fail-safe), et l'architecture logicielle garantit des résultats mathématiquement fiables sur le long terme.

Cependant, ces millions de lignes de coordonnées brutes sont illisibles pour un cerveau humain. Pour comprendre ce qu'il s'est passé, repérer d'éventuelles collisions ou observer le chaos gravitationnel, nous devons donner vie à ces chiffres.

👉 **Direction le Module 06 - Visualisation**

Introduction

Ce module est dédié au programme de post-traitement : un script indépendant dont l'unique objectif est de lire notre fichier CSV d'export, d'en extraire la dynamique et de générer un tableau de bord graphique en deux dimensions.

[6.1] : La Lecture des Données et la Restitution du Temps (Le Parsing)

Contexte : Cibler le fichier de simulation sur le disque, en extraire de manière sécurisée les métadonnées de configuration ainsi que les trajectoires des astres, et isoler l'identité des corps présents sans aucune saisie manuelle de l'utilisateur.

Ressources >

- Héritage** : Le fichier CSV de sortie généré par le moteur au Module 05, contenant la ligne de configuration `# DUREE_SEC:...` et l'en-tête aplati (`Astre_X`, `Astre_Y`).
- Modules Python** : `pandas` pour le chargement de la matrice de données, `sys` et `os` pour la gestion des flux systèmes et des chemins d'accès.

Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme 1 : L'ergonomie de ciblage (Console CLI vs Fichier par défaut)

L'utilisateur doit pouvoir lancer rapidement le visualiseur pour tester un rendu sans retaper un long chemin à chaque fois, tout en conservant la possibilité d'analyser un fichier spécifique via la console.

L'Arbitrage : Nous implémentons une logique d'aiguillage hybride dans `obtenir_chemin_fichier()`. Si un argument est passé en ligne de commande (`sys.argv`), il est prioritaire. À défaut, le script charge automatiquement le fichier standard défini dans la configuration globale (`FICHIER_PAR_DEFAUT`). Un pare-feu système intercepte immédiatement les extensions incorrectes ou l'absence physique du fichier sur le disque dur.

Dilemme 2 : L'extraction d'un fichier à format mixte (Texte + Matrice)

Notre fichier CSV contient sur sa toute première ligne des métadonnées scientifiques cruciales (Durée, Pas, Intervalle) précédées du caractère `#`. Charger le fichier directement via Pandas provoquerait une erreur de parsing car la structure n'est pas homogène.

L'Arbitrage : Nous utilisons une lecture séquentielle. Le script ouvre d'abord le fichier avec un `readline()` natif de Python pour capturer la chaîne de configuration et isoler les variables de timing via un découpage (`split`). Ensuite, nous passons le flux à `pandas.read_csv()` avec le paramètre `comment='#'`. Pandas saute proprement la ligne informative et charge la matrice numérique à la vitesse maximale du processeur.

Dilemme 3 : L'identification des N-Corps et la déduction du temps

Le visualiseur doit être universel et accepter un nombre *N* d'astres inconnu à l'avance, sans que les noms ne soient codés en dur dans le script. De plus, stocker une colonne "Temps" à chaque ligne du CSV alourdirait le fichier sur le disque pour rien.

L'Arbitrage : Pour l'identité des corps, le script scanne dynamiquement les en-têtes de colonnes du DataFrame Pandas et filtre toutes les clés se terminant par `_X` pour reconstruire la liste exacte des astres (`colonne[:-2]`). Pour le temps, aucune donnée n'est stockée sur le disque. Le visualiseur calcule algébriquement le temps physique exact à la volée lors de la lecture d'une ligne en multipliant l'index de l'image par l'intervalle de sauvegarde de la simulation (`frame_index * intervalle_csv`).

Décisions Implémentées :

- Sécurisation du ciblage** : Utilisation de `sys.argv` couplé à un fallback automatique sur `FICHIER_PAR_DEFAUT` avec levée d'erreurs explicites (`.endswith('.csv')`, `os.path.exists`).
- Double lecture du CSV** : Extraction des constantes de simulation via `f.readline()` puis isolation du tableau numérique pur via `pd.read_csv(..., comment='#')`.
- Scan dynamique** : Utilisation d'une compréhension de liste pour isoler le nom des astres à partir des suffixes de coordonnées.
- Restitution temporelle** : Reconstitution mathématique du temps universel à l'écran via le produit `frame_index * intervalle_csv` dans la boucle d'animation.

L'Émergence >

Les données brutes de l'univers sont maintenant chargées en mémoire vive, l'identité des astres est isolée et le chronomètre est étalonné.

Le problème suivant s'enclenche : Si nous injectons directement ces coordonnées brutes dans un repère graphique fixe, les planètes risquent d'être invisibles (soit superposées sur un seul pixel au centre, soit perdues hors de l'écran) en raison des écarts d'échelle gigantesques propres à la gravitation. Comment calibrer automatiquement la fenêtre d'affichage avant le rendu ?

→ Passe au Maillon 6.2 : Le Cadrage Automatique et le Défi de l'Échelle (La Caméra Dynamique).

[6.2] : Le Cadrage Automatique et le Défi de l'Échelle (La Caméra Dynamique)

Contexte : Ajuster automatiquement la fenêtre d'affichage (le zoom de la caméra) pour que tous les astres soient visibles à l'écran, peu importe si le système est microscopique (une fusée) ou macroscopique (le système solaire externe).

Ressources >

- Concept Physique** : L'Unité Astronomique (1.496×10^{11} m), les ordres de grandeur de distance.
- Outils Mathématiques** : Fonctions NumPy `abs()`, `max()`, `log10()` et `ceil()`.

Le Labo (Réflexions & Arbitrage) >

Pourquoi le choix de la 2D ? (L'arbitrage de clarté) : L'espace possède trois dimensions, mais une visualisation en 3D sur un écran plat implique des projections complexes, des distorsions optiques et une perte de repères.

L'arbitrage : Conformément aux recommandations de physique numérique, nous travaillons dans le plan orbital bidimensionnel ($X - Y$). Cela simplifie le tracé, garantit une mesure directe des distances et rend immédiate la vérification visuelle des lois de Kepler (trajectoires elliptiques).

Le piège du zoom fixe (Dompter le gigantisme) : Si nous bloquons la taille de l'écran sur une valeur fixe (par exemple 35 UA), le visualiseur sera parfait pour Uranus, mais si nous chargeons une simulation de laboratoire à petite échelle, l'écran sera vide car le zoom sera trop lointain. À l'inverse, si nous simulons une comète qui s'éloigne, elle sortira instantanément du cadre.

La solution (La caméra adaptative) : Au lancement, le script parcourt l'intégralité du fichier de données et cherche la coordonnée la plus grande en valeur absolue parmi tous les astres et sur toute la durée du calcul (`max_pos`). À partir de ce nombre pur en mètres, le code prend une décision d'unité humaine :

- Si `max_pos` dépasse 1% d'une UA, la caméra bascule en **Unités Astronomiques (UA)**.
- Si elle dépasse 1000 km, elle passe en **kilomètres (km)**.
- En dessous, elle reste en **mètres (m)**.

Pour éviter que les planètes ne collent aux bords du cadre, nous appliquons une marge de sécurité de 10% et arrondissons proprement la limite au demi-ordre de grandeur supérieur grâce à une analyse logarithmique ($10 \times \text{floor}(\log_{10}(\text{marge}))$). La caméra est ainsi parfaitement calibrée avant même la première image.

Décisions :

- Calcul du maximum absolu des données pour définir la taille de la boîte d'affichage.
- Graduation automatique et intelligente des axes (Ticks) en notation scientifique adaptée à l'unité choisie.

[6.3] : Le Tableau de Bord et la Fluidité Graphique

Contexte : Créer une animation fluide tout en affichant un panneau latéral de statistiques (télémétrie) qui s'actualise en temps réel, sans surcharger l'écran ni le processeur.

Ressources >

- **Héritage** : La variable `intervalle` calculée dans le Module 03 et le DataFrame chargé au Module 6.1.
- **Modules Graphiques** : `matplotlib.gridspec` pour le découpage de l'écran, `FuncAnimation` pour la boucle d'images.
- **Concept Informatique** : Le *Blitting* (mise à jour sélective des pixels en mémoire vidéo).

Le Labo (Les Dilemmes d'Ingénierie) >

Dilemme 1 : Le rafraîchissement vs Les limites de Matplotlib Si Matplotlib doit redessiner à chaque image le décor complet (les axes, le fond, le texte), le processeur s'effondre. *L'Arbitrage (Le Blitting)* : Nous activons l'option `blit=True`. Matplotlib prend une "photo" du décor fixe au démarrage. À chaque itération, le code ne lui renvoie que les objets qui ont bougé (planètes, chrono).

Dilemme 2 : L'Avance Rapide (Le Frame Skipping) Si le CSV contient 200 000 lignes, lire l'animation prendrait des heures. Relancer le moteur physique pour changer l'intervalle d'origine serait un gâchis de temps CPU. *L'Arbitrage (L'intervalle visuel dynamique)* : Nous introduisons `SAUT_IMAGES`. En lisant 1 ligne sur 10, on multiplie la vitesse de lecture par 10 au post-traitement, sans toucher à la physique. *La limite physique* : La boucle de Matplotlib a un temps de traitement incompressible d'environ 10 millisecondes. Pour éviter qu'il ne s'engorge, nous verrouillons la demande avec un plafond strict (`min(60, FPS_CIBLE)`) couplé à ce délai minimum de 10 ms.

Dilemme 3 : L'Asphyxie Visuelle (Le cas des N-Corps massifs) Notre générateur de chaos peut créer des univers à 150 astres. Tracer 150 sillages infinis et afficher 150 noms dans une légende rendrait l'écran illisible et saturerait la RAM graphique. *L'Arbitrage (Le nettoyage conditionnel)* : Nous imposons des limites visuelles strictes. La légende est désactivée au-delà de 10 corps. Les traînées orbitales sont bridées à une longueur mémoire courte (`TAILLE_TRAINEE`), avec la possibilité de les rendre complètes uniquement sur demande pour des expériences précises. Enfin, pour que l'utilisateur puisse suivre un astre d'un test à l'autre, on attribue les couleurs via une graine aléatoire fixe.

Décisions :

- **Fluidité** : Utilisation de `blit=True` dans `FuncAnimation` pour le rendu partiel.
- **Vitesse temporelle** : Implémentation du `SAUT_IMAGES` dans le découpage des `frames` de lecture pour simuler une avance rapide.
- **Sécurité matérielle** : Calcul du délai avec `max(10.0, 1000.0 / min(60, FPS_CIBLE))` pour ne jamais saturer Matplotlib.
- **Ergonomie dynamique** : - Masquage automatique de la légende `if len(noms_astres) > 10`.
 - Fixation des couleurs via `np.random.default_rng(42)`.
 - Maintien d'un historique de points glissant pour alléger les traînées (`frame_index - TAILLE_TRAINEE`).

Conclusion

Ce module de post-traitement transforme notre code de calcul brut en un véritable laboratoire virtuel interactif. En assumant le choix d'une architecture séparée (le moteur calcule d'un côté, le visualiseur lit de l'autre), nous avons créé un outil performant et modulable, capable de s'adapter instantanément à toutes les échelles de l'Univers sans jamais brider la physique.

L'univers s'anime désormais de manière fluide à l'écran, les échelles s'adaptent toutes seules, et l'utilisateur garde le contrôle total du temps via la télémétrie.

Cependant, si l'animation est esthétiquement réussie, une question fondamentale demeure : comment prouver que ce que nous voyons à l'écran est **physiquement juste** ? Comment s'assurer que notre intégrateur de Verlet respecte vraiment la théorie, conserve l'énergie et modélise correctement le chaos, prouvant ainsi la validité scientifique de notre code de bout en bout ?

👉 C'est l'ultime étape du projet : Direction le [Module 07 - Analyses](#)

Introduction

La physique a généré ses données (Module 05) et notre moteur graphique a démontré sa capacité à les restituer fidèlement (Module 06). Ce dernier module est consacré à l'exploitation scientifique des simulations produites.

L'objectif n'est plus seulement de calculer des trajectoires, mais de vérifier que notre modèle numérique reproduit correctement les comportements physiques attendus. Les scripts de post-traitement analysent ainsi les données brutes afin d'identifier des propriétés caractéristiques de la dynamique gravitationnelle : stabilité orbitale, conservation des mouvements et apparition du chaos dans le problème à trois corps.

Cette démarche permet de valider indirectement la rigueur de la simulation. Si les phénomènes physiques théoriquement attendus émergent naturellement des données calculées, alors le modèle numérique, l'intégration temporelle de Verlet et l'implémentation vectorielle des forces gravitationnelles peuvent être considérés comme cohérents.

Enfin, la visualisation joue un rôle essentiel d'interprétation : elle permet de confronter directement les trajectoires simulées aux comportements physiques prédits par la théorie, prouvant l'efficacité de notre architecture de résolution séparée (Générateur / Post-Traitement).

[7.1] : Détermination Numérique de la Vitesse de Libération

Contexte & Objectif : Vérifier si notre intégrateur numérique est capable de faire émerger la limite physique d'évasion gravitationnelle (11.2 km/s pour la Terre) de lui-même, uniquement en calculant des forces pas à pas, sans que la formule théorique ne soit jamais écrite dans le code.

📖 Ressources & Connaissances >

- Outils :** `pandas` pour l'extraction de données, `matplotlib.pyplot` pour le tracé analytique statique.
- Données d'entrée :** Huit fichiers CSV distincts (pré-générés dans le dossier `/Outputs/`) simulant des lancements avec des vitesses initiales v_0 allant de 10.0 à 11.5 km/s depuis la surface terrestre ($R = 6378$ km).
- Physique Théorique :** La vitesse de libération (v_l) est le point de bascule où l'énergie mécanique totale du système devient positive, transformant une trajectoire fermée (ellipse) en une trajectoire ouverte. Pour la Terre, $v_l = \sqrt{\frac{2GM}{R}} \approx 11.2$ km/s.

🔗 Le Labo (Réflexions & Arbitrage) >

Le Dilemme de la Comparaison (Le problème des corps multiples) :

L'approche naïve consisterait à placer 8 fusées avec 8 vitesses différentes autour de la Terre dans le même fichier CSV. *L'Arbitrage :* Bien que physiquement possible, lancer les fusées depuis des points différents ruinerait la rigueur de la comparaison orbitale. Pour obtenir une lecture claire de l'ouverture de l'orbite, toutes les trajectoires doivent partager strictement la même origine géométrique. Nous avons donc généré une série de fichiers CSV distincts (un par vitesse) partant du même point à $t = 0$. Le script de post-traitement se charge d'extraire et de superposer ces chronologies parallèles.

L'Automatisation de la Légende :

Afin d'éviter le code en dur, notre script lit chaque fichier, isole la ligne à $t = 0$, et extrait la vitesse initiale (V_0) directement depuis la donnée brute. C'est cette valeur qui génère automatiquement la légende du graphique. Afin de ne masquer aucune trajectoire, la légende est déportée géométriquement à l'extérieur de la zone de dessin.

Décisions Analytiques :

- Développement d'un script superposant les données de multiples fichiers CSV.
- Tracé de la Terre à l'échelle stricte ($R = 6378$ km) à l'origine du repère pour identifier visuellement l'orbite de départ.
- Mise à l'échelle de toutes les distances en "Mégamètres" (10^6 mètres) pour éviter d'asphyxier les axes de lecture.

✓ Bilan Analytique (Confirmation Théorique)

Le tracé graphique obtenu démontre visuellement la transition géométrique : les courbes correspondant aux vitesses < 11.2 km/s se referment (retour sur Terre), tandis que celles ≥ 11.2 km/s se courbent sans jamais revenir (trajectoire ouverte). L'émergence naturelle de cette limite physique valide formellement la conservation et la distribution énergétique opérée par notre moteur N-Corps.

💡 Perspective d'amélioration (Évolutivité) :

Dans le cadre de ce projet, le script a été ciblé textuellement sur les colonnes `Fusee` et `Terre`. Dans une version ultérieure, ce script d'analyse pourrait être rendu totalement générique : le code scannerait l'en-tête du CSV pour identifier lui-même l'attracteur central et le projectile, permettant l'analyse de trajectoires d'évasion sur n'importe quel système planétaire.

[7.2] : L'Étude du Chaos (Le Problème à 3 Corps)

Contexte & Objectif : Démontrer l'instabilité fondamentale de la gravitation pour un système à plus de deux corps, et mettre en évidence la sensibilité extrême aux conditions initiales (l'effet papillon). L'objectif n'est pas de mesurer la longévité orbitale absolue, mais de prouver mathématiquement qu'une erreur infinitésimale grandit de manière exponentielle au fil du temps.

📖 Ressources & Connaissances >

- Outils :** Script `chaos.py` (`pandas`, `numpy`, `matplotlib`).
- Données d'entrée :** Deux fichiers CSV contenant 3 étoiles de masse solaire (1.989×10^{30} kg) placées en triangle équilatéral. Dans le second fichier, la vitesse initiale d'une étoile est modifiée (ajout de 0.0002 m/s).
- Physique Théorique :** La prévisibilité d'un système à plus de deux corps est impossible sur le long terme en raison de sa nature chaotique. Nous utilisons l'écart relatif (Δr) entre deux univers jumeaux pour isoler ce comportement.

🔗 Le Labo (Réflexions & Arbitrage) >

Le placement des astres (La méthode scientifique) :

Obtenir un système où 3 corps tournent sur une orbite circulaire exige que l'attraction gravitationnelle compense exactement la force centrifuge. Nous les avons placées aux sommets d'un triangle équilatéral parfait. Les vecteurs vitesses ont été orientés tangentiellement au cercle circonscrit, avec une précision absolue de 10 décimales pour garantir la symétrie à $t = 0$.

Le Paradoxe du système "Stable" :

Malgré cette perfection initiale, notre simulation de référence finit par se disloquer.

La Diagnostic : La solution en triangle équilatéral pour trois masses égales est une configuration de Lagrange. Le critère de Gascheau démontre qu'elle n'est stable que si un corps est largement plus massif que les autres. Pour trois masses égales ($\mu = 3$), le mouvement est linéairement instable par nature. L'algorithme de Verlet n'invente pas le chaos, mais il introduit inévitablement des erreurs de discrétisation spatiale et d'arrondi des flottants. Ce "bruit numérique" casse la symétrie, et l'instabilité naturelle du système amplifie cette erreur jusqu'à la dislocation.

La Comparaison des Univers :

Notre script aligne les deux fichiers de résultats temporellement, calcule la distance mutuelle entre chaque paire d'étoiles, et applique le calcul d'écart relatif en pourcentage : $\Delta r = \frac{r_{chaos} - r_{stable}}{r_{stable}} \times 100$.

Décisions Analytiques :

- Calcul de la configuration initiale de Lagrange.
- Développement du script d'analyse isolant l'écart relatif (Δr) avec synchronisation temporelle automatique des deux simulations (pour pallier l'arrêt d'une des simulations par le système anticollision).
- Double approche de validation : l'analyse visuelle de la déviation exponentielle croisée avec l'audit direct des données brutes (CSV) pour dater avec précision la rupture des systèmes.

✓ Bilan Analytique (Validation de l'Effet Papillon)

Le tracé graphique obtenu valide parfaitement le comportement chaotique attendu :

1. **La phase dormante** : La courbe reste plate à 0% pendant plus de 8 ans. Les deux univers évoluent de manière identique, l'anomalie n'ayant pas encore pris le dessus.
2. **L'explosion exponentielle** : La courbe s'emballe de manière verticale, prouvant que des conditions initiales quasi identiques mènent irrémédiablement à une perte de corrélation spatiale.

Les limites de la courbe (La nécessité de la donnée brute) : Le graphique de déviation relative ne permet pas d'identifier directement la perte de stabilité du système de référence. Une fois le système perturbé devenu erratique, la courbe devient irrégulière et masque le comportement de l'autre univers. Les véritables dates de rupture de stabilité ($t \approx 8, 29$ ans pour le système perturbé et $t \approx 13, 56$ ans pour le système de référence) ont donc été déterminées exclusivement par notre outil de visualisation et par l'analyse directe des distances mutuelles dans les données brutes (CSV).

[7.3] : Validation à Long Terme (Le Système Solaire sur 165 ans)

Contexte & Objectif : Prouver la viabilité et la stabilité numérique de notre moteur physique. Pour cela, nous simulons le Système Solaire complet (le Soleil et ses 8 planètes) sur une durée de 165 ans. Ce laps de temps n'est pas choisi au hasard : il correspond exactement à une révolution complète de Neptune, la planète la plus éloignée de notre système.

📖 Ressources & Connaissances >

- **Outils** : Le moteur `simulation.py` et le script d'animation `visualiseur.py` (aucun script d'analyse supplémentaire n'est requis).
- **Données d'entrée** : Un fichier `systeme_solaire.csv` contenant les masses, positions (sur X) et vitesses initiales (sur Y) réelles de nos 8 planètes.
- **Physique Théorique** : L'algorithme de Verlet est dit "symplectique". Contrairement à la méthode d'Euler, l'énergie totale oscille très légèrement mais ne dérive jamais sur le long terme.

🔧 Le Labo (Réflexions & Arbitrage) >

L'Épreuve du Temps (Le Crash-Test) : L'un des plus grands défis de la simulation numérique est la dérive de l'énergie. Si notre intégrateur n'était pas parfait, les erreurs de calcul s'accumuleraient à chaque pas de temps (chaque seconde simulée). Sur 165 ans, les planètes finiraient inévitablement par s'écraser sur le Soleil ou s'échapper dans le vide.

Le Paradoxe Visuel des Échelles : En gravitation, les différences d'échelle sont immenses. Mercure orbite à $\sim 5.79 \times 10^{10}$ mètres du Soleil, tandis que Neptune évolue à $\sim 4.495 \times 10^{12}$ mètres.

Le Diagnostic : Si nous dézoomons la caméra pour voir Neptune, les planètes telluriques (Mercure, Vénus, Terre, Mars) se retrouvent toutes superposées dans un seul pixel au centre de l'écran.

Décisions Analytiques :

- Lancement de la simulation via le moteur vectorisé (Mode Matrices) sur une durée de 165 années (`unite = 'A'`) pour absorber la charge colossale de calculs.
- Injection directe du fichier CSV de sortie dans le `visualiseur.py`.
- Utilisation de la fonction "Caméra Adaptative" du visualiseur pour absorber les écarts d'échelle planétaires.
- Utilisation du visualiseur pour générer la vidéo de l'évolution, mettant en évidence la stabilité du système ainsi que la clôture de l'orbite de Neptune.

✓ Bilan Analytique (Validation Symplectique)

L'animation graphique produite valide magistralement notre moteur : malgré des milliards d'opérations mathématiques effectuées par le processeur, Neptune clôture parfaitement son ellipse à l'écran après 165 ans, sans aucune dérive apparente (comme le reste du système). Ce résultat confirme la nature symplectique de notre intégrateur temporel.

[7.4] : La Stabilité Absolue (L'Orbite en Huit de Moore)

Contexte & Objectif :

Isoler et prouver la perfection géométrique de notre moteur physique (l'intégration symplectique de Verlet). Pour cela, nous devons utiliser une configuration à trois corps de masses égales qui soit mathématiquement capable de résister aux erreurs d'arrondis de l'ordinateur, afin de démontrer que notre algorithme ne dérive pas sur le très long terme.

📖 Ressources & Connaissances >

- **Outils** : Le moteur physique `simulation.py` et le script d'animation `visualiseur.py`.
- **Données d'entrée** : Un fichier `3_soleils_huit.csv` contenant trois astres de masse solaire (

1.989×1030 kg1.989×1030 kg

) placés avec des coordonnées et des vecteurs vitesses extrêmement précis.
- **Physique Théorique** : L'orbite en lemniscate (figure en 8). Découverte numériquement par Cris Moore en 1993 et prouvée mathématiquement par Chenciner et Montgomery en 2000, c'est une chorégraphie gravitationnelle où trois corps se poursuivent indéfiniment sur la même courbe fermée.

🔧 Le Labo (Réflexions & Arbitrage) >

Le Mythe du Triangle (Le Théorème de Gascheau) : Nous avons vu (Module 7.2) que la configuration en triangle équilatéral parfait finit systématiquement par exploser au bout de quelques années. Ce n'est pas un défaut de notre code, mais une loi physique (Théorème de Gascheau, 1843) : avec 3 masses égales, cet équilibre est sur le fil du rasoir. La moindre erreur d'arrondi des flottants (la limite matérielle de la machine) suffit à réveiller le chaos.

Le Dilemme : Comment prouver alors que notre algorithme est robuste sur le long terme, si la configuration de test de base est elle-même instable par nature ?

La Solution en Huit (Le Bassin d'Attraction) : Pour contourner cette limite théorique, nous avons fait appel à une solution beaucoup plus rare : l'orbite en "Huit". L'avantage majeur et unique de cette solution est que son "bassin d'attraction" est large. Contrairement au triangle, elle tolère les micro-perturbations. Elle s'auto-corrige naturellement et encaisse les erreurs d'arrondis inévitables de l'ordinateur sans se disloquer.

L'Épreuve de vérité pour notre Moteur : Si notre intégrateur mathématique générerait la moindre friction numérique ou ajoutait artificiellement de l'énergie (comme le ferait la méthode d'Euler), même l'orbite en Huit finirait par se briser. La tester sur une longue durée est le juge de paix de notre code.

Décisions Analytiques :

- Création d'un fichier de configuration spécial (`3_soleils_huit.csv`) traduisant les constantes mathématiques de Moore adaptées aux échelles de notre système.
- Exécution de la simulation sur une durée de 50 ans via le moteur matriciel, avec un pas de temps de haute précision.

- Utilisation du `visualiseur.py` avec l'option de traînée complète (`TRAINEE_COMPLETE = True`) pour tracer visuellement le "8" parfait à l'écran et s'assurer que les orbites ne s'épaississent pas au fil du temps.

✓ Bilan Analytique (La Preuve Symplectique Ultime)

En soumettant cette orbite à notre moteur, le résultat visuel est sans appel : la trajectoire reste parfaitement stable, fluide et strictement superposée à elle-même au-delà de 50 ans simulés. Cette démonstration est la preuve ultime que notre algorithme de Velocity-Verlet préserve parfaitement l'énergie géométrique du système. Nous avons atteint une précision et une rigueur satisfaisante, sans avoir eu recours à des algorithmes de calcul beaucoup plus lourds en temps d'exécution (comme Runge-Kutta d'ordre 4).